

MODUL 14

(GRAPH II)



DESKRIPSI TEMA

- Travelling Salesman Problem
- Single-Source Shortest Path Problem
- Dijkstra's algorithm
- Minimum Spanning Tree
- Kruskal's Algorithm
- Prim's Algorithm

CAPAIAN PEMBELAJARAN MINGGUAN (SUB-CAPAIAN PEMBELAJARAN)

Mahasiswa dapat memahami dan menerapkan konsep minimum spanning trees (C_3).

PENUNJANG PRAKTIKUM

1. Software CodeBlocks

LANGKAH-LANGKAH PRAKTIKUM

Tutorial 1 – Travelling Salesman Problem

TSP merupakan suatu algoritma penyelesaian masalah untuk mencari jalan terpendek dari suatu node source dan kembali lagi ke node source.

Salinlah dan pahami kode di bawah ini.

```

1  #include<stdio.h>
2
3  int ary[10][10],completed[10],n,cost=0;
4
5  void takeInput(){
6      int i,j;
7
8      printf("Enter the number of villages: ");
9      scanf("%d",&n);
10
11     printf("\nEnter the Cost Matrix\n");
12
13     for(i=0;i < n;i++){
14         printf("\nEnter Elements of Row: %d\n",i+1);
15
16         for( j=0;j < n;j++){
17             scanf("%d",&ary[i][j]);
18
19             completed[i]=0;
20         }
21
22         printf("\n\nThe cost list is:");
23
24         for( i=0;i < n;i++){
25             printf("\n");
26
27             for(j=0;j < n;j++){
28                 printf("\t%d",ary[i][j]);
29             }
30         }
31     }
32
33     void mincost(int city)
34     {
35         int i,ncity;
36
37         completed[city]=1;
38
39         printf("%d-->",city+1);
40         ncity=least(city);
41
42         if(ncity==999){
43             ncity=0;
44             printf("%d",ncity+1);
45             cost+=ary[city][ncity];
46
47             return;
48         }
49         mincost(ncity);
50     }

```

```

51
52 int least(int c){
53     int i,nc=999;
54     int min=999,kmin;
55
56     for(i=0;i < n;i++){
57         if((ary[c][i]!=0)&&(completed[i]==0))
58             if(ary[c][i]+ary[i][c] < min){
59                 min=ary[i][0]+ary[c][i];
60                 kmin=ary[c][i];
61                 nc=i;
62             }
63     }
64
65     if(min!=999)
66         cost+=kmin;
67
68     return nc;
69 }
70
71 int main(){
72     takeInput();
73
74     printf("\n\nThe Path is:\n");|
75     mincost(0); //bernilai 0 karena merupakan starting vertex
76
77     printf("\n\nMinimum cost is %d\n ",cost);
78
79     return 0;
80 }

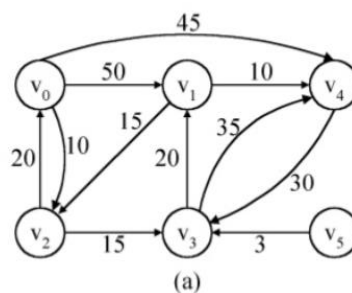
```

Tutorial 2 – Single Source Shortest Path

Single Source Shortest Path (SSSP) merupakan algoritma pencarian jalan terpendek dari satu source ke node tujuan yang diinginkan. Prinsipnya, ketika mengecek node satu per satu, maka nilai jaraknya akan diakumulasi dari sourcenya.

The single-source shortest path problem

- shortest paths from v_0 to all destinations

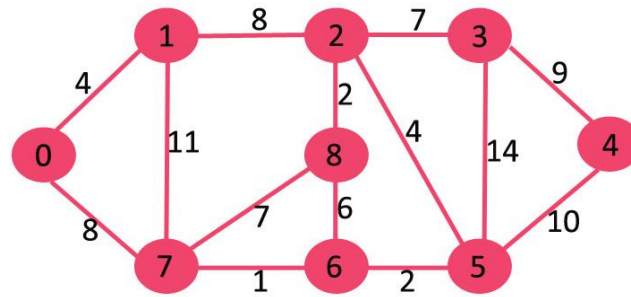


	Path	Length
1)	$v_0 v_2$	10
2)	$v_0 v_2 v_3$	25
3)	$v_0 v_2 v_3 v_1$	45
4)	$v_0 v_4$	45

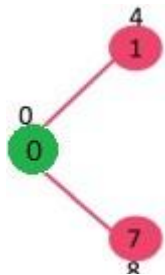
(b)

Tutorial 3 – Dijkstra Algorithm

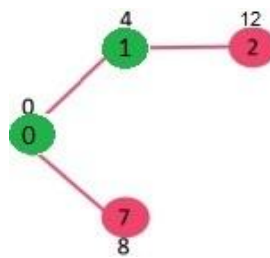
Algoritma Dijkstra merupakan algoritma untuk menentukan jarak atau lintasan terpendek dari sebuah graph. Setiap tahap dari algoritma ini akan menentukan jarak yang paling minimum dari source atau asalnya dengan cara mengakumulasi total jarak yang telah ditempuh dari source saat tiba di suatu node.



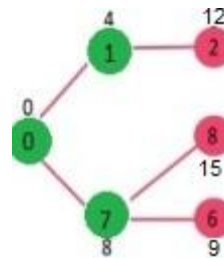
Tahap Algoritma Dijkstra



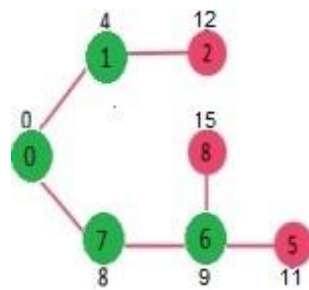
Tahap 1



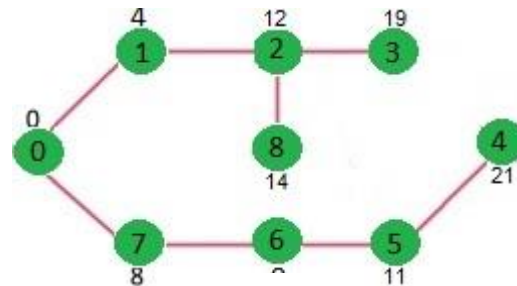
Tahap 2



Tahap 3



Tahap 4



Tahap 5

Silahkan Anda salin dan pahami kode di bawah ini.

```
#include <stdio.h>
#include <stdlib.h>

// Jumlah vertex
#define V 9
#define INT_MAX 999999

// Function untuk menemukan vertex dengan nilai minimum dari
// himpunan vertex yang tidak termasuk dalam path tree
int minDistance(int dist[], int sptSet[])
{
    // Inisialisasi min value
    int min = INT_MAX, min_index;
    int v;
    for (v = 0; v < V; v++)
        if (sptSet[v] == 0 && dist[v] <= min)
            min = dist[v], min_index = v;

    return min_index;
}

// Function untuk print distance array yang telah dibuat
int printSolution(int dist[], int n)
{
    int i;
    printf("Vertex    Distance from Source\n");
    for (i = 0; i < V; i++)
        printf("%d \t %d\n", i, dist[i]);
}

// Function yang mengimplementasikan algoritma Dijkstra
// Function ini menggunakan representasi adjacency matrix
void dijkstra(int graph[V][V], int src)
{
    int i, v, count, dist[V]; // Output array.
    // Jarak src ke i

    int sptSet[V]; // sptSet[i] akan bernilai true jika vertex i merupakan jalan terpendek
    // Inisialisasi semua jarak sebagai nilai INFINITE and sptSet[] bernilai salah pada awalnya
    for (i = 0; i < V; i++)
        dist[i] = INT_MAX, sptSet[i] = 0;

    // Jarak dari source ke dirinya sendiri selalu bernilai 0;
    dist[src] = 0;

    // Menemukan Shortest Path
    for (count = 0; count < V-1; count++){
        int u = minDistance(dist, sptSet);
        sptSet[u] = 1;

        for (v = 0; v < V; v++)
            if (!sptSet[v] && graph[u][v] && dist[u] != INT_MAX
                && dist[u]+graph[u][v] < dist[v])
                dist[v] = dist[u] + graph[u][v];
    }
    printSolution(dist, V);
}

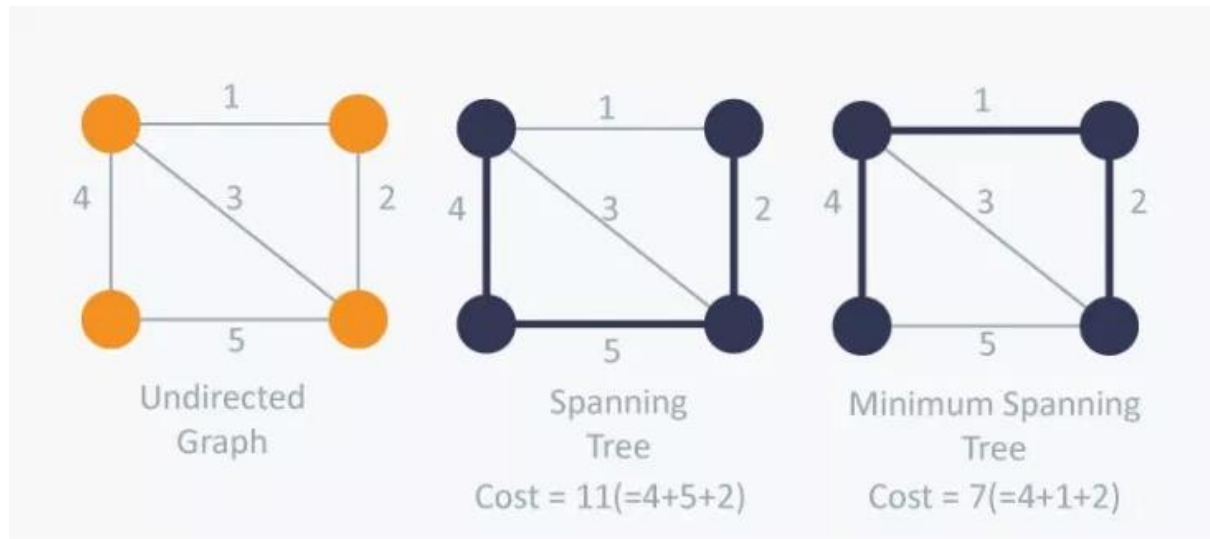
int main()
{
    int graph[V][V] = {{0, 4, 0, 0, 0, 0, 0, 8, 0},
                       {4, 0, 8, 0, 0, 0, 0, 11, 0},
                       {0, 8, 0, 7, 0, 4, 0, 0, 2},
                       {0, 0, 7, 0, 9, 14, 0, 0, 0},
                       {0, 0, 0, 9, 0, 10, 0, 0, 0},
                       {0, 0, 4, 14, 10, 0, 2, 0, 0},
                       {0, 0, 0, 0, 0, 2, 0, 1, 6},
                       {8, 11, 0, 0, 0, 0, 1, 0, 7},
                       {0, 0, 2, 0, 0, 0, 6, 7, 0}};

    dijkstra(graph, 0);

    return 0;
}
```

Tutorial 3 – Minimum Spanning Tree

Minimum spanning tree adalah tree dengan semua edge nya terhubung namun tidak membentuk sebuah siklus(cycle). Dalam minimum spanning tree jumlah weight yang didapatkan adalah yang paling minimum dari seluruh edge yang sudah terhubung. Ada 2 algoritma yang terkenal untuk menemukan minimum spanning tree yakni Algoritma Kruskal dan Algoritma Prim.



Tutorial 4 – Kruskal's Algorithm

Algoritma Kruskal membentuk spanning tree dengan menambahkan satu persatu edge menjadi spanning tree. Algoritma Kruskal mengikuti pendekatan *greedy*, yakni pada setiap iterasi, algoritma tersebut mencari edge dengan bobot paling kecil dan menambahkannya ke *growing spanning tree*.

Langkah Algoritma:

- Sortir edge graf berdasarkan bobot mereka.
- Mulai tambahkan edge ke MST dari edge dengan bobot terkecil hingga edge dengan bobot terbesar.
- Hanya tambahkan edge yang tidak membentuk siklus, yaitu edge yang hanya terhubung ke komponen yang terputus.

Salinlah potongan kode dibawah ini dan pahami.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  //struct untuk merepresentasikan sisi dengan bobot di dalam graph
6  struct Edge
7  {
8      int src, dest, weight;
9  };
10
11 //struct untuk merepresentasikan graph yang terkoneksi, tidak berarah dan memiliki bobot
12 struct Graph
13 {
14     // V-> jumlah dari simpul, E-> jumlah dari sisi
15     int V, E;
16
17     // graph direpresentasikan sebagai array of edges
18     // Selama graph tersebut tidak berarah, sisi
19     // dari sumber ke tujuan juga merupakan sisi
20     // dari tujuan ke sumber. Keduanya terhitung sebagai 1 sisi disini
21     struct Edge* edge;
22 };
23
```

```
24 // Membuat graph dengan V simpul dan E sisi
25 struct Graph* createGraph(int V, int E)
26 {
27     struct Graph* graph = (struct Graph*) malloc(sizeof(struct Graph));
28     graph->V = V;
29     graph->E = E;
30
31
32     graph->edge = (struct Edge*) malloc(sizeof(struct Edge) * E);
33
34     return graph;
35 }
36
37 // Struct untuk merepresentasikan subset untuk union-find
38 struct subset
39 {
40     int parent;
41     int rank;
42 };
43
44 // fungsi untuk menemukan set dari elemen i
45 int find(struct subset subsets[], int i)
46 {
47     // find root and make root as parent of i
48     // (path compression)
49     if (subsets[i].parent != i)
50         subsets[i].parent = find(subsets, subsets[i].parent);
51
52     return subsets[i].parent;
53 }
```

```

55 // fungsi untuk menggabungkan dua set dari x dan y
56 void Union(struct subset subsets[], int x, int y)
57 {
58     int xroot = find(subsets, x);
59     int yroot = find(subsets, y);
60
61     // hubungkan tree dengan rank yang lebih kecil dibawah tree dengan rank yang lebih besar
62     if (subsets[xroot].rank < subsets[yroot].rank)
63         subsets[xroot].parent = yroot;
64     else if (subsets[xroot].rank > subsets[yroot].rank)
65         subsets[yroot].parent = xroot;
66
67     // Jika rank nya sama, maka pilih salah satu sebagai root
68     // dan ranknya bertambah 1
69     else
70     {
71         subsets[yroot].parent = xroot;
72         subsets[xroot].rank++;
73     }
74 }
75
76 // Badingkan dua sisi sesuai dengan bobot mereka
77 // Gunakan qsort() untuk melakukan sorting array of edges
78 int myComp(const void* a, const void* b)
79 {
80     struct Edge* a1 = (struct Edge*)a;
81     struct Edge* b1 = (struct Edge*)b;
82     return a1->weight > b1->weight;
83 }
84

```

```

85 // Fungsi utama untuk membangun MST menggunakan Algoritma Kruskal
86 void KruskalMST(struct Graph* graph)
87 {
88     int V = graph->V;
89     struct Edge result[V]; // Ini akan menyimpan hasil dari MST
90     int e = 0; // Index variabel, digunakan untuk result[]
91     int i = 0; // Index variabel, digunakan untuk edges yang sudah diurutkan
92
93     // Step 1 : urutkan semua sisi yang ada berdasarkan bobot mereka.
94     // Jika tidak boleh mengubah graph yang ada, maka dapat dibuat
95     // salinan dari array of edges
96     qsort(graph->edge, graph->E, sizeof(graph->edge[0]), myComp);
97
98     // Alokasi memori untuk membuat V subset
99     struct subset *subsets =
100         (struct subset*) malloc( V * sizeof(struct subset) );
101
102     // Buat B subset dengan single element
103     int v;
104     for (v = 0; v < V; ++v)
105     {
106         subsets[v].parent = v;
107         subsets[v].rank = 0;
108     }

```



```

110 // Jumlah dari sisi yang akan diambil sama dengan V-1
111 while (e < V - 1)
112 {
113     // Step 2 : Pilih sisi terkecil. Dan
114     // tambahkan index untuk iterasi selanjutnya
115     struct Edge next_edge = graph->edge[i++];
116
117     int x = find(subsets, next_edge.src);
118     int y = find(subsets, next_edge.dest);
119
120     // Jika menambahkan sisi ini tidak membuat siklus,
121     // tambahkan sisi ini ke dalam result dan tambahkan
122     // index dari result untuk sisi selanjutnya
123     if (x != y)
124     {
125         result[e++] = next_edge;
126         Union(subsets, x, y);
127     }
128     // Else buang sisi selanjutnya
129 }
130
131 // tampilkan isi dari result[] untuk
132 // menampilkan MST yang telah dibuat
133 printf("Following are the edges in the constructed MST\n");
134 for (i = 0; i < e; ++i)
135     printf("%d -- %d == %d\n", result[i].src, result[i].dest,
136           result[i].weight);
137 return;
138 }

```

```

141 int main()
142 {
143     /* Mari kita buat graph berbobot berikut
144     |      10
145     0-----1
146     | \      |
147     6|  \5   |15
148     |   \   |
149     2-----3
150     |   4   */
151     int V = 4; // Jumlah dari simpul pada graph
152     int E = 5; // Jumlah dari sisi pada graph
153     struct Graph* graph = createGraph(V, E);
154
155     // tambahkan sisi 0-1
156     graph->edge[0].src = 0;
157     graph->edge[0].dest = 1;
158     graph->edge[0].weight = 10;
159

```

```
161 // tambahkan sisi 0-2
162 graph->edge[1].src = 0;
163 graph->edge[1].dest = 2;
164 graph->edge[1].weight = 6;
165
166 // tambahkan sisi 0-3
167 graph->edge[2].src = 0;
168 graph->edge[2].dest = 3;
169 graph->edge[2].weight = 5;
170
171 // tambahkan sisi 1-3
172 graph->edge[3].src = 1;
173 graph->edge[3].dest = 3;
174 graph->edge[3].weight = 15;
175
176 // tambahkan sisi 2-3
177 graph->edge[4].src = 2;
178 graph->edge[4].dest = 3;
179 graph->edge[4].weight = 4;
180
181 KruskalMST(graph);
182
183 return 0;
184 }
```

Tutorial 5 – Prim's Algorithm

Algoritme Prim juga menggunakan pendekatan *greedy* untuk menemukan *minimum spanning tree*. Dalam algoritme Prim, kita mengembangkan spanning tree dari posisi asal. Bagian yang berbeda dari algoritme Kruskal adalah ditambahkannya edge ke growing spanning tree; sedangkan pada algoritme Prim, verteks yang ditambahkan ke growing spanning tree.

Salinlah potongan kode di bawah ini dan pahami.

```
1  #include <stdio.h>
2  #include <limits.h>
3  #include <stdbool.h>
4  // Jumlah dari simpul pada graph
5  #define V 5
6
7  // Fungsi untuk menemukan vertex dengan
8  // minimum key value, dari kumpulan simpul
9  // yang belum termasuk dalam MST
10 int minKey(int key[], bool mstSet[])
11 {
12     // Inisialisasi min value
13     int min = INT_MAX, min_index;
14     int v;
15     for (v = 0; v < V; v++)
16         if (mstSet[v] == false && key[v] < min)
17             min = key[v], min_index = v;
18
19     return min_index;
20 }
21
22 // Fungsi untuk menampilkan
23 // MST yang sudah dibuat dan disimpan di parent[]
24 int printMST(int parent[], int n, int graph[V][V])
25 {
26     printf("Edge \tWeight\n");
27     int i;
28     for (i = 1; i < V; i++)
29         printf("%d - %d \t%d \n", parent[i], i, graph[i][parent[i]]);
30 }
31
```

```

32 // Fungsi untuk membangun dan menampilkan MST
33 // untuk representasi graph menggunakan adjacency matrix
34 void primMST(int graph[V][V])
35 {
36     // Array untuk menyimpan MST yang dibangun
37     int parent[V];
38     // Key values untuk mengambil bobot minimum sisi
39     int key[V];
40     // Untuk representasi kumpulan dari simpul yang belum termasuk kedalam MST
41     bool mstSet[V];
42
43     // Inisialisasi semua keys sebagai tak terhingga
44     int i;
45     for (i = 0; i < V; i++)
46         key[i] = INT_MAX, mstSet[i] = false;
47
48     // Selalu masukkan vertex pertama ke dalam MST
49     // Buat key 0 sehingga vertex ini terambil sebagai vertex pertama
50     key[0] = 0;
51     parent[0] = -1; // Node pertama selalu sebagai root dari MST
52
53     // MST akan mempunyai V simpul
54
55     int count;
56     for (count = 0; count < V-1; count++)
57     {
58         // Pilih minimum key vertex dari kumpulan
59         // simpul yang belum termasuk ke dalam MST
60
61         int u = minKey(key, mstSet);
62
63         // Tambahkan vertex yang diambil ke dalam kumpulan MST
64
65         mstSet[u] = true;
66
67         // Update key value dan index parent dari
68         // simpul yang berdekatan dari vertex yang dipilih.
69         // Hanya dari simpul yang belum termasuk ke dalam MST
70         int v;
71         for (v = 0; v < V; v++)
72
73             // graph[u][v] bukan nol hanya untuk simpul yang berdekatan dari m
74             // mstSet[v] yang bernilai false untuk simpul yang belum termasuk ke dalam MST
75             // Update key hanya jika graph[u][v] lebih kecil dari key[v]
76             if (graph[u][v] && mstSet[v] == 0 && graph[u][v] < key[v])
77                 parent[v] = u, key[v] = graph[u][v];
78     }

```

```

80     // tampilkan MST yang sudah dibangun
81     printMST(parent, V, graph);
82 }
83
84
85 int main()
86 {
87     /* Mari kita buat graph berikut
88         |      2      3
89         (0)--(1)--(2)
90         |    /  \    |
91         6| 8 /    \ 5 |7
92         | /      \ |
93         (3)------(4)
94         |      9      */
95     int graph[V][V] = {{0, 2, 0, 6, 0},
96                       {2, 0, 3, 8, 5},
97                       {0, 3, 0, 0, 7},
98                       {6, 8, 0, 0, 9},
99                       {0, 5, 7, 9, 0}};
100
101     // Tampilkan solusi
102     primMST(graph);
103
104     return 0;
105 }

```

Challenge

1. Pada tutorial Travelling Salesman Problem (TSP), kode yang diberikan masih menggunakan global variable. Silahkan anda ubah kode yang ada pada tutorial TSP, sehingga tidak menggunakan global variable.

Format penamaan: **NIM_NAMA_Challenge01_Week14.c**

2. Buatlah program yang dapat menentukan jarak minimum dari kota asal ke kota tujuan. Dengan rute 1 arah saja (undirected graph). Untuk menentukan jalur antara 1 kota ke kota lainnya, dilakukan dengan menggunakan inputan dengan format Source#Destination#Weight. Source dan destination berupa huruf a-z, dan weight berupa integer. Inputan akan berhenti ketika anda memasukkan salah satu kota yang anda masukkan berupa "-", contohnya -#-#o atau -#a#o. (Clue: Gunakan Dijkstra Algorithm)

Contoh input:

[source]#[destination]#[weight]

a#b#2

a#c#5

b#c#3

b#d#7

c#e#6

c#d#1

b#a#3

-#-#0

Source: a

Contoh Output:

Untuk mencapai kota a dari kota a membutuhkan jarak terpendek 0.

Untuk mencapai kota b dari kota a membutuhkan jarak terpendek 2.

Untuk mencapai kota c dari kota a membutuhkan jarak terpendek 5.

Untuk mencapai kota d dari kota a membutuhkan jarak terpendek 6.

Untuk mencapai kota e dari kota a membutuhkan jarak terpendek 11.

3. Buatlah sebuah program Minimum spanning tree dengan menggunakan adjacency matrix. Program akan meminta inputan berupa adjacency matrix. Program memiliki 2 menu utama yaitu algoritma Kruskal dan algoritma Prim. Output program berupa edges yang dipilih berdasarkan setiap algoritma tersebut dan minimum cost dari total edges yang dipilih. Contoh tampilan program dapat dilihat pada file pendukung.

Format penamaan : **NIM_NAMA_Challenge03_Week14.c**

REFERENSI

1. Modul Struktur Data Tahun 2017
2. https://www.slideshare.net/Krish_ver2/51-greedy
3. <https://wirasetiawan29.wordpress.com/2015/04/02/tentang-algoritma-dijkstra/>
4. <https://www.techiedelight.com/single-source-shortest-paths-dijkstras-algorithm/>
5. <https://www.geeksforgeeks.org/dijkstras-shortest-path-algorithm-greedy-algo-7/>
6. <https://www.geeksforgeeks.org/travelling-salesman-problem-set-1/>
7. <https://indonesia.hackerearth.com/minimum-spanning-tree-pohon-rentangan-minimum/>
8. <https://www.geeksforgeeks.org/kruskals-minimum-spanning-tree-algorithm-greedy-algo-2/>
9. <https://www.geeksforgeeks.org/prims-minimum-spanning-tree-mst-greedy-algo-5/>